Principal Types

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss **principle types** and **specialization**
- Go though some examples of HM^- type inference
- Demo an implementation of HM^-

Practice Problem

$$\cdot \vdash \lambda x\,.\,xx : \tau \dashv \mathscr{C}$$

Determine the type τ and constraints $\mathscr{C}$ such that the above judgment is derivable

Answer

$$\cdot \vdash \lambda x\,.\,xx : \tau \dashv \mathscr{C}$$

Recap

Recall: Unification

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to \mathsf{int} \to \mathsf{int} &\doteq b \to c
\end{aligned}
$$

Unification is the process of solving a system of equations over *symbolic* expressions

Recall: ADT Unification Problem

A **unification problem** is a collection of equations of the form

$$
\begin{aligned}
s_1 &\doteq t_1 \\
s_2 &\doteq t_2 \\
&\vdots \\
s_k &\doteq t_k
\end{aligned}
$$

where $s_1, \ldots, s_k$ and $t_1, \ldots, t_k$ are **terms** (values of an ADT with variables)

Recall: Type Unification Problem

$$\sigma ::= \mathsf{int} \mid \mathsf{bool} \mid \alpha \mid \sigma \to \sigma$$

A **type unification problem** is a collection of equations of the form

$$\begin{aligned} s_1 &\doteq t_1 \\ s_2 &\doteq t_2 \\ &\vdots \\ s_k &\doteq t_k \end{aligned}$$

where $s_1, \ldots, s_k$ and $t_1, \ldots, t_k$ are types

Recall: Unifiers

Recall: Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

Recall: Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

Recall: Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

Recall: Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

$$\begin{aligned} \mathscr{S} t_1 &= \mathscr{S} s_1 \\ \mathscr{S} s_2 &= \mathscr{S} t_2 \\ &\vdots \\ \mathscr{S} s_k &= \mathscr{S} t_k \end{aligned}$$

Recall: Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

$$\begin{aligned} \mathscr{S} t_1 &= \mathscr{S} s_1 \\ \mathscr{S} s_2 &= \mathscr{S} t_2 \\ &\vdots \\ \mathscr{S} s_k &= \mathscr{S} t_k \end{aligned}$$

where $\mathscr{S} t = [t_n/x_n] \ldots [t_1/x_1] t$

Recall: Most General Unifiers

Recall: Most General Unifiers

A **most general unifier** is a solution $\mathcal{S}$ such that, for any solution $\mathcal{S}'$, there is another solution $\mathcal{S}''$ such that $\mathcal{S}'\tau = \mathcal{S}''\mathcal{S}\tau$

Recall: Most General Unifiers

A **most general unifier** is a solution $\mathcal{S}$ such that, for any solution $\mathcal{S}'$, there is another solution $\mathcal{S}''$ such that $\mathcal{S}'\tau = \mathcal{S}''\mathcal{S}\tau$

In other words, $\mathcal{S}'$ is $\mathcal{S}$ *with more substitutions*

An Algorithm (Pseudocode)

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰
```

An Algorithm (Pseudocode)

in: type unification problem $\mathscr{U}$
out: most general unifier for $\mathscr{U}$

$\mathscr{S} \leftarrow$ empty solution

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty
```

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty

  MATCH eq:
```

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty

  MATCH eq:

    (S) t ≐ t ⟹ 𝒰 ← 𝒰\{eq}  // if syntactically equal then remove eq
```

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty

  MATCH eq:

    (S) t ≐ t ⟹ 𝒰 ← 𝒰\{eq}  // if syntactically equal then remove eq

    (F) s₁ → t₁ ≐ s₂ → t₂ ⟹ 𝒰 ← 𝒰\{eq} ∪ {s₁ ≐ s₂, t₁ ≐ t₂} // remove eq and add s₁ ≐ s₂ and t₁ ≐ t₂

    (A) α ≐ t or t ≐ α where α ∉ FV(t) ⟹ // type variable α does not appear free in t
```

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

    OTHERWISE $\implies$ **FAIL**

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

    OTHERWISE $\implies$ **FAIL**

RETURN $\mathcal{S}$

Another Practice Problem

$$\beta \doteq \eta$$
$$\alpha \to \beta \doteq \alpha \to \gamma$$
$$\alpha \to \beta \doteq \gamma \to \eta$$
$$\alpha \to \beta \doteq \mathsf{int} \to \eta$$

Determine a most general unifier to the above type unification problem using the algorithm we just gave

Answer

$$\beta \doteq \eta$$

$$\alpha \to \beta \doteq \alpha \to \gamma$$

$$\alpha \to \beta \doteq \gamma \to \eta$$

$$\alpha \to \beta \doteq \mathsf{int} \to \eta$$

Principal Types

Recall: Hindley-Milner

$$
\begin{aligned}
e ::= \; & \lambda x \,.\, e \mid ee \\
& \mid \text{let } x = e \text{ in } e \\
& \mid \text{if } e \text{ then } e \text{ else } e \\
& \mid e + e \mid e = e \\
& \mid n \mid x \\
\sigma ::= \; & \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma \\
\tau ::= \; & \sigma \mid \forall \alpha \,.\, \tau
\end{aligned}
$$

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Constraints say what must hold so e is well-typed

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Constraints say what must hold so e is well-typed

The idea: We're formalizing the process of "collecting" constraints, as in our example

Recall: HM$^-$ (Typing)

$$\frac{n \texttt{ is an integer}}{\Gamma \vdash n : \mathsf{int} \dashv \varnothing} \ \texttt{(int)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \Gamma \vdash e_3 : \tau_3 \dashv \mathscr{C}_3}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau_3 \dashv \tau_1 \doteq \mathsf{bool}, \tau_2 \doteq \tau_3, \mathscr{C}_1, \mathscr{C}_2, \mathscr{C}_3} \ \texttt{(if)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 = e_2 : \mathsf{bool} \dashv \tau_1 \doteq \tau_2, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(eq)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 + e_2 : \mathsf{int} \dashv \tau_1 \doteq \mathsf{int}, \tau_2 \doteq \mathsf{int}, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(add)}$$

$$\frac{\alpha \texttt{ is fresh} \qquad \Gamma, x : \alpha \vdash e : \tau \dashv \mathscr{C}}{\Gamma \vdash \lambda x . e : \alpha \to \tau \dashv \mathscr{C}} \ \texttt{(fun)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \alpha \texttt{ is fresh}}{\Gamma \vdash e_1 e_2 : \alpha \dashv \tau_1 \doteq \tau_2 \to \alpha, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(app)}$$

Aside: Let-Expressions

HM$^-$ (Typing Let-Expressions)

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2 \dashv \mathscr{C}_1, \mathscr{C}_2}\ \texttt{(let)}$$

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem". This rule doesn't allow let-functions to be polymorphic!

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem". This rule doesn't allow let-functions to be polymorphic!

There are some <u>debates</u> in PL regarding let-polymorphism...

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem". This rule doesn't allow let-functions to be polymorphic!

There are some debates in PL regarding let-polymorphism...

The Takeaway. We'll treat typing of top-level let-expressions as *different* from local let-expressions

Principle Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Principle Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraints $\mathscr{C}$ defined a *unification problem.* Given an MGU $\mathscr{S}$ we can get the "actual" type of e:

Principle Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraints $\mathscr{C}$ defined a *unification problem*. Given an MGU $\mathscr{S}$ we can get the "actual" type of e:

$$\text{principal}(\tau, \mathscr{C}) = \forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau \text{ where } \mathsf{FV}(\mathscr{S}\tau) = \{\alpha_1, \ldots, \alpha_k\}$$

Principle Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraints $\mathscr{C}$ defined a *unification problem.* Given an MGU $\mathscr{S}$ we can get the "actual" type of e:

$$\text{principal}(\tau, \mathscr{C}) = \forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau \text{ where } \text{FV}(\mathscr{S}\tau) = \{\alpha_1, \ldots, \alpha_k\}$$

i.e, the **principal type** of e (**note:** it may not exist). Every type we *could* give e is a *specialization* of $\forall \alpha_1, \ldots, \alpha_k . \mathscr{S}\tau$

Example

Determine the principal type of $\lambda f.\lambda x.f(x+1)$

Example

Show that $\mathsf{let}\ f = \lambda x\,.\,x\ \mathsf{in}\ f\ (f\ 2 = 2)$ *has no principal type*

Putting everything together

Putting everything together

Input: program P (sequence of top-level let-expressions)

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression let $x = e$ in P:

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\mathsf{let}\ x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\mathsf{let}\ x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\text{let } x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)

3. *Generalization:* Quantify over the free variables in $\mathscr{S}\tau$ to get the principle type $\forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau$ of e

Putting everything together

Input: program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\text{let } x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable
2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)
3. *Generalization:* Quantify over the free variables in $\mathscr{S}\tau$ to get the principle type $\forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau$ of e
4. Add $(x : \forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau)$ to Γ

As a Type System

$$\frac{}{\Gamma \vdash \epsilon}\ \texttt{(emptyProg)}$$

$$\frac{\Gamma \vdash e : \tau \dashv \mathscr{C} \qquad \Gamma, x : \mathsf{principal}(\tau, \mathscr{C}) \vdash P}{\Gamma \vdash \mathsf{let}\ x = e\ P}\ \texttt{(topLet)}$$

We can also express this as a type system with judgments of the form $\Gamma \vdash P$, where P is a program (note there is no ":")

Example

```
let id = fun x -> x
let _ = id (id 2 = 2)
```

Example

```
let id = fun x -> x
let _ = id (id 2 = 2)
```

$\varnothing \vdash$ `let id = fun x -> x let a = id (id 2 = 2)` (topLet)

Example

```
let id = fun x -> x
let _ = id (id 2 = 2)
```

$\varnothing \vdash$ `let id = fun x -> x let a = id (id 2 = 2)` (topLet)

$\varnothing \vdash$ `fun x -> x` $: \alpha \to \alpha \dashv \varnothing$ (...)

$\ldots$ (...)

$\{$`id` $: \forall \alpha . \alpha \to \alpha\} \vdash$ `let a = id (id 2 = 2)`

Example

```
let id = fun x -> x
let _ = id (id 2 = 2)
```

$\varnothing \vdash$ `let id = fun x -> x let a = id (id 2 = 2)` (topLet)

- $\varnothing \vdash$ `fun x -> x` $: \alpha \to \alpha \dashv \varnothing$ (...)
  - ... (...)
- $\{$`id` $: \forall \alpha . \alpha \to \alpha\} \vdash$ `let a = id (id 2 = 2)` (topLet)
  - $\{$`id` $: \forall \alpha . \alpha \to \alpha\} \vdash$ `id (id 2 = 2)` $: \alpha \dashv \mathscr{C}$ (...)
    - ... (...)
  - $\{$`id` $: \forall \alpha . \alpha \to \alpha$, `a` : `bool`$\} \vdash \epsilon$

Example

```
let id = fun x -> x
let _ = id (id 2 = 2)
```

$\varnothing \vdash$ `let id = fun x -> x let a = id (id 2 = 2)` (topLet)

- $\varnothing \vdash$ `fun x -> x` $: \alpha \to \alpha \dashv \varnothing$ (...)
  - ... (...)
- $\{$`id` $: \forall \alpha . \alpha \to \alpha\} \vdash$ `let a = id (id 2 = 2)` (topLet)
  - $\{$`id` $: \forall \alpha . \alpha \to \alpha\} \vdash$ `id (id 2 = 2)` $: \alpha \dashv \mathscr{C}$ (...)
    - ... (...)
  - $\{$`id` $: \forall \alpha . \alpha \to \alpha$, `a` : `bool`$\} \vdash \epsilon$ (emptyProg)

demo

(HM⁻)

Summary

The **principal type** of an expression is the most general type we could give it

Specialization defines a partial ordering on type schemes from most to least general

Our unification algorithm gives us a most general unifier, which will always give us the principle type of an expression